

Playing Knit Graphs

Emily Quimby

School of Computing Science
Sir Alwyn Williams Building
University of Glasgow
G12 8QQ

A dissertation presented in part fulfilment of the requirements of the Degree of Master of Science at The University of Glasgow

September 16, 2020

Abstract

This paper addresses how computer programming can be used as a tool for producing art whilst addressing issues around femininity, algorithms for domestic living and women's roles in computing, including the making of electronic music. Computers and computer programming have been used as tools for creating art since they first came in to prominence during the mid-C20th. In those early years, much of the work with computers was done by women, a fact that was largely forgotten as the importance of computers grew in the latter part of the C20th. More recently, this information has been uncovered and written about by a number of different people including those working in academia. The interest in yarn crafts and feminism has seen a resurgence in recent years too. This has included an interest in the relationship between computers and crafts such as knitting, and the similarities between the two.

The program developed as part of this project sought to bring several strands of enquiry together: Feminine Crafts; Machines; Programming Algorithms; Electronic Music; Women in Tech. The resulting concept for the work was to develop a program that translates knitting patterns into sound. This was done by rigorously analysing how a knitting pattern could be translated and by interpreting knit-structures as an abstract data type, a KnitGraph. Three main algorithms were developed, represented by the KnitGraph methods doKnitting(), doUnpicking() and dropAStitch(). These processes were hijacked in order to auto-generate unique soundscapes. The work was consistently tested and a process of evaluation was undertaken in order to identify ways in which the program could be improved. This helped form ideas for future work and ways the project as a whole could be developed further. These evaluations also helped to establish a real interest in the project and opened up discussions around how computer programming can be used to explore other related ideas in a creative way.

Education Use Consent

I hereby give my permission for this project to be shown to other University of Glasgow students and to be distributed in an electronic format. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Name: _____ Signature: _____

Acknowledgements

Thanks go to my supervisor Dr Jess Enright, especially for supporting my ideas for this project and allowing me the freedom to pursue them. Also, for generally being a constant, reassuring and chilled out presence throughout. Thanks to my husband Joseph for doing his bit to support me over the past year.

Special dedication to my mam Julia, who encouraged me to do the course in the first place ”I wish I was doing it!”. I love you.

Contents

1 Introduction **1**

2 Context & Analysis **2**

2.1 Software as Art 2

2.2 Women and Machines 3

2.3 Is Knitting Cool? 4

2.4 Concept for Knit-Programming 4

3 Requirements **5**

3.1 Epic One: Feminine Crafts, a User Interface 6

3.2 Epic Two: Programming Crafts as Coded Algorithms 6

3.3 Epic Three: Music Making Machine 6

4 Design & Implementation **7**

4.1 The GUI 8

4.2 Reading in KnitSpeak files 9

4.3 The Knit Graph 10

4.4 The Sound Machine 11

4.5 Future-Proofing & Limitations 12

5 Testing **13**

5.1 Modelling the KnitGraph for Testing 13

5.2 Informal Progressive Testing 14

5.3 Unit Testing using JUnit 14
5.4 Other Considerations 15

6 Evaluation **16**

6.1 The Art Crit 17
6.2 A Software Engineers Perspective 17
6.3 Evaluation Summary 18

7 Conclusion **19**

Appendix A Class Diagram **21**

Appendix B KnitSpeak File Parsing **22**

B.1 Example KnitSpeak files 22
B.2 Pseudocode and Workings for KnitSpeakIn 24

Appendix C Sample Workings for KnitGraph **25**

C.1 Creating the dropAStitch() Algorithm 25

Appendix D Ethics Checklist **27**

Chapter 1

Introduction

The ways in which individual artists produce their work is not often revealed, and in that sense, the creative process invites curiosity. Each artist has their own individual *process*. This may change over time, but it is largely a consistent way for them to work. For example, a painter may make fine layers of paint, then carve away at the dried surface before flooding it again with more paint, and finally wiping away the excess to reveal the finished work. This *process* can be repeated to make a series of paintings. Sometimes the process isn't about actual *making*, but about research and collating ideas before drawing or making plans about what happens next. Almost always it is about solving some problem or other. This *process* is an algorithm for making art, a set of steps the artist must take in order to produce results. It's up to the artist to develop this *algorithm* just like it is up to the computer programmer to develop their algorithms for solving given problems.

This project seeks to use computer programming as a tool, in much the same way as an artist might use paint and brushes. The aim is to explore the poetic potential of computer programming and to raise questions about ”function” and ”usefulness” when it comes to software development. With this in mind, it's also seen as an opportunity to write a program that serves the purpose of being ”fun” both to implement and to engage with. The challenge is to develop a set of complex algorithms for producing the artwork, by translating knitting patterns into sound. Knitting patterns are, after all, an *algorithm* for recreating knitted objects.

This project has been approached as something with the potential to grow and develop over time and into the future. A thorough period of research helped to establish the concept for the project and discover ways to implement it. As a minimum, the project seeks to develop a program that can translate a range of knitting patterns into some abstract data type such as a graph or singly-linked list, and that that could in some way be used to create a composition of sound that was unique to a given pattern. This paper describes the process of researching, making, testing and analysing the program, and goes on to explore how the project as a whole could be developed beyond its deadline.

Chapter 2

Context & Analysis

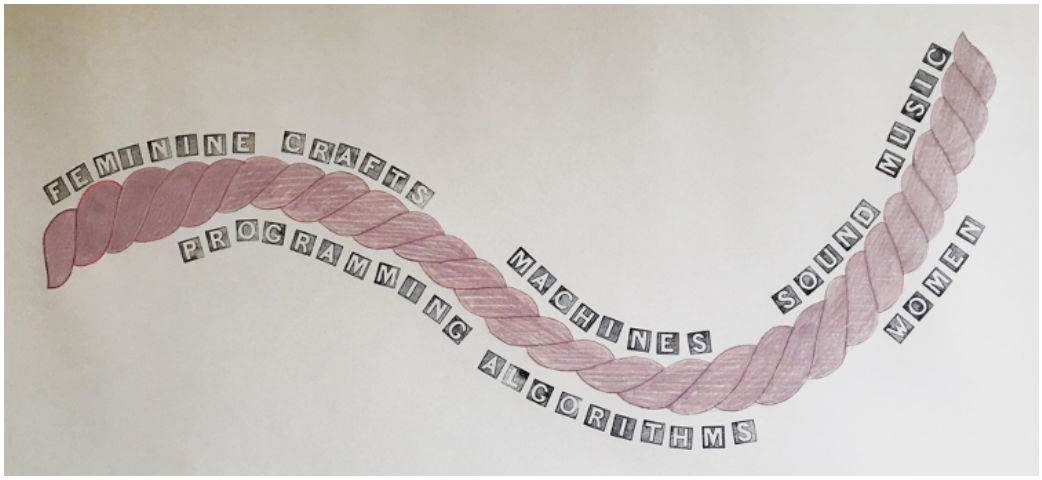


Figure 2.1: Drawing of the concept: Interacting with ”crafted” materials to produce sound.

Several strands of enquiry have been knitted together to form the basis of this project. These strands are broadly illustrated in Figure. 2.1 which was drawn as part of the initial research. The rest of this chapter aims to further cement the research and interests that have formulated the concept for this project.

2.1 Software as Art

Since the 1950s, computers have been used as tools for artistic expression. At that time, computers were not as accessible as they are today, and so their creative potential was explored by a limited number of people - namely computer scientists and mathematicians [28]. From drawing machines such as the ones programmed by Frieder Nake [29], to video works such as the ‘Oscillon’ series by Ben Laposky [9], and early examples of electronic music such as ‘Illiac Suite’ by Lejaren Hiller and Leonard Isaacson [19][18], computers touched the artistic output of a wide range of artists and scientists working in various modalities at that time. By 1968, this trend had been recognised by

Jasia Reichardt who curated the landmark exhibition of Computer Art "Cybernetic Serendipity" at the Institute for Contemporary Art (ICA) in London [5]. This exhibition represented a wide range of artists, musicians, scientists and polymaths who were working creatively to explore the potential of computers. Since then, the creative output revolving around computers has continued to grow. More recently, examples of computer art can be seen through the output of Corey Arcangel [25] and Olia Lialina [2]. Both have been producing digital artworks since the 1990s and are recognised internationally for their contribution to the wider artistic community. Parallels can be drawn between the development of the computer as fine-art medium and other areas of artistic expression too. The seed of the digital DIY movement demoscene [20] was planted in the 1970s when home computers first became available. This reflects other singularly artistic DIY initiatives such as the punk movement of the 70s and 80s, zine-culture, post-punk, the rave scene and so on. It seems clear then that as art-culture has developed in the post-war era and beyond, so has computer-culture, and for much of the time, there has been overlap and reflection between the two, each providing the other a means to an end of one sort or another. This relationship has been crystalised by the many degree courses and research centres in existence today, both in the UK and internationally, such as MFA Computational Arts at Goldsmiths in London, BA(Hons) Computer Arts at Abertay University and the (research) Institute of Creative Technologies at De Montford University. There are also grass-roots organisations such as the artist-run School for Poetic Computation in New York, whose primary concern is to "explore the intersections of code, design, hardware and theory — focusing especially on artistic intervention."

2.2 Women and Machines

Famously, Ada Lovelace recognised the potential for Charles Babbage's Analytical Engine (inspired by jacquard looms and their use of punch cards), to work for any kind of data, and not be limited to only mathematical calculations [10]. In so doing, she predicted modern computing as we know it today. Not so widely understood is the broader role of women, in the early days of modern computing. There are many examples of the important role that women played, particularly in the 1940s and 50s. This was a time when men were largely absent at war, and necessity trumped discrimination when it came to work. It's also a time when "computing" was seen as low-skilled work, work suitable for a woman. For example, the women that worked on the ENIAC computer in the 1940s [11], at Bletchley Park as code-breakers on the Colossus computer, for NASA as "computers" (now known as "programmers") and who made a significant contribution to winning the space race between the 1930s and 1960s [26]. As men returned to join the work-force at home and the importance of computers became ever-increasingly apparent, men took over the jobs of their female counterparts, and the, now considered *highly skilled*, work that those women had done was largely forgotten. It's only in recent years that recognition for those women has been regained [14]. The phenomenon of forgotten women working in tech can be seen in music too. Delia Derbyshire was a once forgotten, and now revered, pioneer of electronic music. Likewise Wendy Carlos, Bebe Baron and Daphne Oram [16]. This level of gender-imbalance in tech still exists today. If you google "women in tech" you are not short on articles that seek to describe the problem, or initiatives that seek to address it. There is no such thing as "men in tech".

2.3 Is Knitting Cool?

In recent years, as computers have become ever-increasingly-present in our (domestic) lives, so too has there been an increased interest in feminism and (domestic) crafts such as knitting and weaving. This can be seen through the rise of popular feminist literature [27], and the scores of crafters that have flooded sites like Instagram and Etsy, in particular those that specialise in crafts such as knitting [12]. Although knitting is not limited to the female experience [24], it is particularly of interest given it's strong association with the feminine and domestic. Fibre artists, including those working in knit, represent part of the wider art community: Judy Chicago, Sheila Hicks and more recently Tracey Emin to name just a few [13]. The influence of fibre arts can also be seen in the works of artists like Eva Rothschild and Lousie Bourgeouis. A theme that brings all of these artists together is feminism and a concern around domesticity and handicrafts.
From a computer science perspective, yarn crafts are deeply knitted into the fabric of computers, from the jacquard looms that inspired modern computing to the more contemporary work of physicist Elisabetta Matsumoto who says "knitting is coding, and yarn is a programmable material" [22]. Matsumoto is not alone in her assertions on the relationship between knitting and code. Many others have made similar associations and have developed projects based on the similarities between knitting and coding. Knit2Code is an initiative developed by Lindive Matlali in South Africa that seeks to teach young people how to code through knitting [17]. Knit-Picking is a group project [15] that seeks to translate hand-knit patterns via a KnitGraph data structure into machine-knitted objects. There are other academic papers on a similar subject too, like this one by Chelsea Battell [3], which establishes a new language "Purl", a coded version of standard kntting files (KnitSpeak).

2.4 Concept for Knit-Programming

Translation is a common theme between art, crafts and computer programming. Art translates ideas into tangible artefacts, knitters translate patterns into knitted objects, computer programmers translate the real-world into useful abstractions. In the 1950s, Christopher Strachey programmed the Manchester University computer to auto-generate love letters [21], more recently the enigmatic "Shawn & Yoshimi" of `http://www.lullatone.com/` have created the app "typatone" which translates a users written words into music. These are interesting examples of where computer programming has been put to use for the purposes of poetry, to evoke a feeling from the user/s or audience. They transcend mere "usefulness". Inspired by these (amongst others) works, this project aims to address concerns around female representation in computing through the medium of knit-translation in a way that is poetic in nature. An initial plan was made to translate KnitSpeak files into an appropriate abstract data type and then develop some output based on this. The ouput could be any number of things, light, sound or printed patterns to name just a few, but to honour the aforementioned female pioneers of electronic music, it was decided that sound would be the most appropriate output.

Chapter 3

Requirements

The question is: how can concerns around the representation of women in computing be expressed poetically through software development? In an attempt to address some of the ideas expressed in Chapter 2, we will look at how yarn (arguably, traditionally a medium associated with women) can be interpreted through computation representing the act of knitting. Furthermore, we will extend this idea to reflect women's roles in making electronic music, another way that women have (and still are) engaged with programming. More specifically, the software seeks to interpret knitting patterns as algorithms written in Java which will, in turn, form the basis of some sound output.

The overall aim is to use code as a medium for producing an artwork, and as such, there is no clearly defined customer. As with artworks in general, the intention is to raise more questions than are answered. They are poetic in nature, and so for this software, the aim is to achieve something imaginative, sensitive and emotive. Naturally, consideration is given to the artwork's "audience", but interaction with the system will be kept minimal, so as to allow the program to speak for itself. In that respect, this is a non-typical software development project. Typically, the approach to requirement-gathering would be to identify the customer and gain feedback from them in order to understand what functionality they want to be available in the product. There may be a need to consider how a wide range of people from different backgrounds wish to use the software, however, in our case, requirements are understood to be the result of rigorous research into the identified themes of the work (see Chapter 2). Each part of the software is carefully and thoughtfully considered in terms of these themes. Figure 2.1 gives a broad sense of the flow and connection of ideas, and how this may be translated into the flow of the system. Consider it a kind of abstract *use case* diagram.

As a single person designing and implementing this project, it does not seem entirely appropriate to use an approach such as agile, however there are some useful tenets of this approach that can be applied here. For example, we can break down the problem into smaller chunks, which we have done here using the MOSCOW technique to prioritise the workload. These smaller tasks can be shared into larger chunks of work called "epics". Each of these "epics" are seen as a single iteration which takes place over a planned period of time depending on its size. These "epics" were largely tackled in the order outlined below, however some adjustments were needed across the whole program as development proceeded.

Note: ”Must haves” are the minimum requirements for the program to be able to run whilst having sufficient and meaningful functionality to address the question posed at the start of this chapter. ”Should haves” are requirements that if implemented, would improve the user experience. ”Could haves” are also options for improving the user experience, although important, decisions on these can be made at a later date. ”Would haves” are ideas for future work.

3.1 Epic One: Feminine Crafts, a User Interface

- **must** have a working interface
- **must** have the ability to select ”KNIT” to begin knitting process.
- **must** have the ability to select ”UNPICK” to begin process of unpicking stitches once the knitting process has started.
- **must** have the ability to select ”DROP A STITCH” to begin the process of dropping a stitch.
- **must** be able to complete the knitting process and ”PLAY” back the resulting knitted data structure.
- **should** be able to visualise the knitting process.
- **could** be hosted online.
- **could** be available only in a gallery as part of an exhibition.
- **could** be able to select/choose from a range of knitted items by clicking on an image or drop down list.
- **would** be able to auto generate knitting pattern data by scraping sites such as Ravelry.com for free patterns.

3.2 Epic Two: Programming Crafts as Coded Algorithms

- **must** interpret knitting patterns as an appropriate data structure using appropriate set of algorithms which replicate the act(s) of knitting through code.

3.3 Epic Three: Music Making Machine

- **must** make use of knitting algorithms as a way of stimulating sound production.

Chapter 4

Design & Implementation

The main focus of this project is to recreate the knitting process in code and find a way of hijacking the resulting algorithms and data structure to produce an interesting and relevant soundscape. In order to achieve that, it is necessary for the container of such a concept to be kept simple so as to let the algorithms themselves take centre stage. The three elements/epics of the program could easily be divided up into 3 distinct classes, and so (give-or-take), that is how the coding proceeded. The code was planned thus (For a detailed class diagram see Appendix A):

- a GUI class.
- a class for reading in KnitSpeak files.
- a class (KnitGraph) that defines the abstract knitting data structure and the algorithms for building (knitting) and decomposing (unpicking) it.
- a class that contains a series of pre-recorded sounds to be played responsively to the knitting process.
- any additional work for testing and future-proofing (scalability).

Although the program is small, it was still important to make sure that it upheld good software engineering practices. Each class has a distinct role within the program and care was taken to ensure that there was low coupling along with this high cohesion. The overall design was based on the Model-View-Controller pattern [7]. For the most part, its implementation therein was successful, however, it was identified that the viewer of the work would likely benefit from seeing how the KnitGraph model was working behind the scenes. In order to update the text fields of the GUI, whilst the various processes were running, and in order to maintain the simplicity of the program, it was decided that the GUI would be updated as part of the KnitGraph processing. This necessitated a two-way relationship between the view/controller (GUI) and the model (KnitGraph). A decision had to be made between maintaining the integrity of the M-V-C pattern, and allowing the viewer a window into the workings of the program without compromising on the simplicity of the code. Ultimately the decision was made to improve the viewer experience. Of course, there are other options available in terms of how the program could be presented in an art context which will be discussed as part of the evaluation (see Chapter 6) and considerations around future work (see Chapter 7).

4.1 The GUI

The GUI was created using the JavaSwing API. Since it is intended to be displayed either online or in a gallery setting with minimum user input, it was designed to be full-screen and to have a fairly minimal aesthetic. There are three buttons, each one triggers a SwingWorker thread which runs an aspect of the program, and a text field that displays information about what is happening behind the scenes. Figures 4.1 and 4.2 show an early prototype of the GUI and the final design. A text field was added later in response to feedback during the evaluation process.

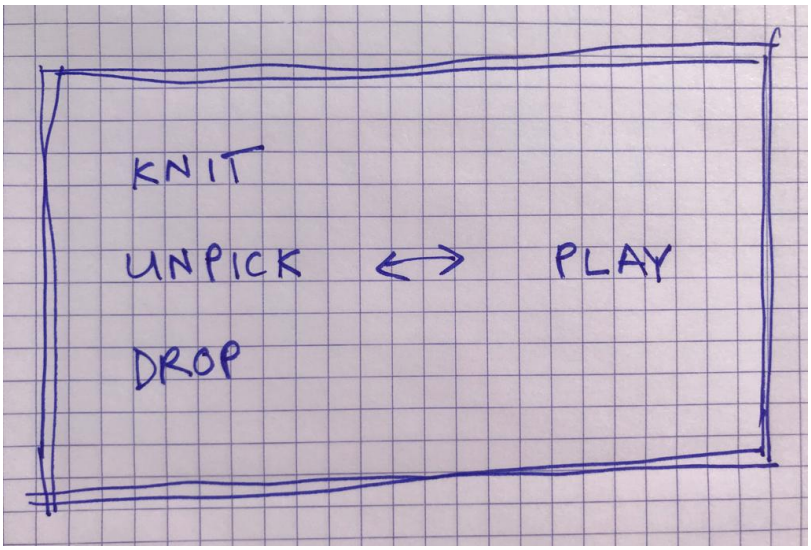


Figure 4.1: Prototype of Simple Interface Design



Figure 4.2: Final GUI Design (note: this image is not visible in greyscale)

As in real-life, you can't knit, unpick and drop stitches all at the same time, so the buttons had to be carefully coordinated so that, for example, whilst "knitting" is in process, we can't begin knitting from the start again, but we could pause "knitting" and start to "unpick" stitches, until we decide we want to "knit" again. In this respect, the organisation of the GUI buttons plays an important role in maintaining the consistency of the program and the KnitGraph (we will see more on this in Section 4.3). Each part of the knitting process is contained within its own SwingWorker thread

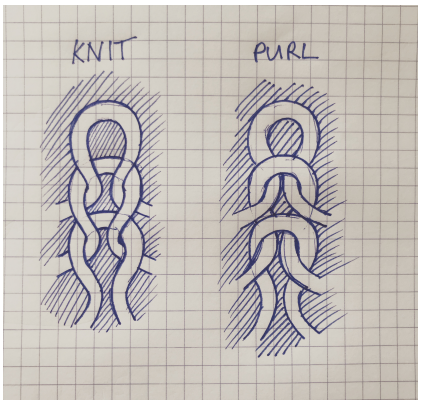
so that these processes can run independently of the GUI. This prevents the GUI from freezing up, and therefore enables us to easily switch from ”knitting” to ”unpicking” to ”knitting” again, to ”dropping stitches” and so on. Once the pattern has been completely knitted, or a stitch has been dropped, the knitting process is finished, and we are able to ”play” the resulting KnitGraph. A pseudo-button ”PLAY” appears in place of ”UNPICK” and all other buttons disappear. Once the KnitGraph has been played, the program resets and we can start the whole process again. The GUI stores a number of KnitSpeak files which are chosen at random when the program begins and again when it resets.

4.2 Reading in KnitSpeak files

A KnitSpeak file is a series of instructions that tell a knitter how to proceed with knitting a given pattern. Each command in a given file translates as a sequence of movements the knitter must make. For example, ”k” translates to ”knit a stitch”, which is the process of inserting a loop of yarn into the working stitch in the row below. ”k” may be followed by a number which will determine the number of stitches to be worked. Some instructions result in increasing or decreasing the number of stitches in a given row, such as ”k2tog”, which means to knit two stitches in the row below together and results in a single new stitch, thus decreasing the number of stitches. Typically, these commands are expressed one row per new line, and each command is separated by a comma. The first row will be a command to ”cast on” followed by the number of stitches, and the final row will be a command to ”cast off”, meaning to cast-off all remaining stitches. The basic language and structure of a KnitSpeak file is roughly the same across the board, however, there can sometimes be slight variations which can make parsing a KnitSpeak file a complex affair. For example, one file might request that you repeat a section of commands by using asterisks and state the number of times it is to be repeated, whereas others may ask you to repeat a set of commands until you have a certain number of stitches left in a row. The number defined at this point is crucial in determining what to do with a given set of commands, however it means two quite different things across two different sets of commands. In order to address this discrepancy, the files which were selected for the program were edited by hand so that a consistent format was achieved (Note that to address scalability, this is a job that could potentially be done by the program itself, however it was not deemed a priority at this stage). From thereon any given KnitFile could be processed by the program to be all lower case, each command added to an array (in order), and any repeated commands inserted into the array at an appropriate point. It is then up to the KnitGraph to iterate through the resulting array of knit commands in order to process them accordingly. This was a fairly straightforward bit of code, however it took some work to produce the algorithm for addressing sets of commands that must be repeated. Illustrations and pseudocode for this part of the program can be found in Appendix B along with some examples of KnitSpeak files.
KnitSpeak instructions were sourced from the site `www.stitch-maps.com` [6]. Cherry-Stitch, Myrtle-Leaf-Stitch and Small-Hearts-Stitch were chosen specifically for their relative simplicity and (mostly) uniformity. More complex files could be added once the basic ground-work of the code had been done. The code was designed to be easily updated. Furthermore, smaller KnitSpeak files would be easier to test and locate bugs if needed.

4.3 The Knit Graph

A piece of knitted cloth is a series of stitches which join together in a variety of ways and in a range of directions. In that respect, a knitted object most closely resembles the Graph Abstract Data Type (ADT). Stitches are nodes/vertices, and the yarn that holds it all together is a series of edges (See Figure 5.1). It also has similarities to a Singly Linked List. Each stitch is a node on a single thread which connects it to the next node in a single direction. A stitch (with some exceptions) is located within the stitch below it. This can be illustrated by looking at the most basic stitches ”knit” or ”purl”. Both are a simple loop inserted into the stitch below, the difference being whether or not the new stitch (loop) is inserted into the front or the back of the stitch below. This could also be seen as a node within a node (See Figure 4.3a).



(a) Sketch of Knit and Purl Stitches.



(b) Modelling Dropped Stitches in Real Life

To further explore this KnitGraph concept, sketches were drawn in order to visualise the knitted structure. In Figure 4.4, we can see a series of nodes joined together with edges. Each time a node (stitch) is ”worked” (knitted into), a reference to the new node is stored inside the ”worked” node. If stitches are increased, multiple stitch-nodes are stored in the ”worked” stitch-node and if stitches are decreased, the new stitch-node is stored in multiple ”worked” stitches. Some commands are anomalous in that they don’t ”work” a stitch below, but exist in their own right as a wrapped stitch or ”yarn over” (yo) stitch. Here a new stitch is created by bringing the yarn over the needle before working the next stitch, and there is no need to store a reference to this new stitch-node as it hasn’t been ”worked” into a stitch below.

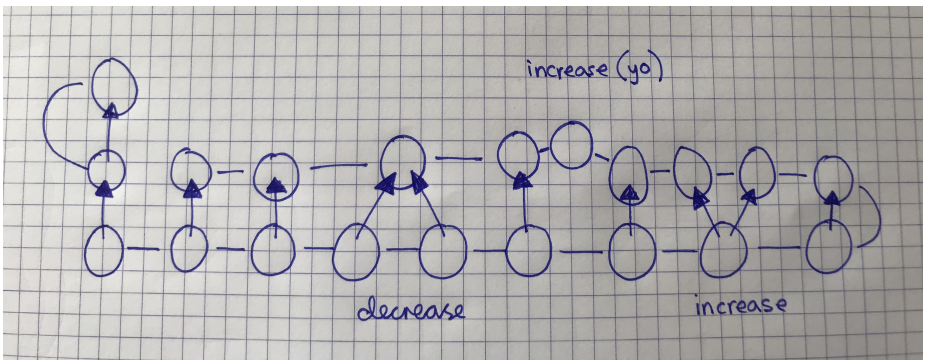


Figure 4.4: Sketch of Knit Graph. Demonstrates what happens when stitches are increased/decreased.

Once the basic structure of the Knit Graph had been established, knit commands (as prepared when reading in the KnitSpeak file) could be divided into a set number of instructions for working the graph. These are as follows:

- **Cast On:** Adds a given number of stitches (nodes), and therefore edges to the graph.
- **Make a Stitch:** Works a single stitch into the stitch below i.e. adds a reference to the new stitch (node) in the stitch below.
- **Increase:** Works multiple stitches into the stitch below i.e. adds multiple references to the new stitch to the stitch below.
- **Decrease:** Works one stitch into multiple stitches below i.e. adds a reference to the new stitch to multiple stitches below.
- **Yarn Over:** Creates a new stitch without working any stitches below. Results in an increase of one stitch.
- **Cast Off:** Tells us when the Knit Graph is complete.

There are three identified broad actions that can take place on a Knit Graph: ”Knitting”; ”Unpicking”; ”Dropping a Stitch”. Knitting is when we create the Knit Graph from a set of commands, the process of adding stitches and therefore edges to the graph. Unpicking is when we pause knitting to remove stitches. Stitches, and therefore edges, are systematically removed from the graph. Dropping a stitch is when a stitch either drops off the (non-working) needle, or is left uncaught by a stitch above it. Dropping a stitch results in the stitches directly beneath the dropped stitch also dropping, and the stitches below that dropping and so on. Visually, this leaves a laddered effect in the knitted cloth, which in order to fix could mean starting the whole process of knitting the graph again. See Figure 4.3b and Appendix C, Figures C.1 and C.2.
Each of these three actions is represented in the code by three distinct algorithms. An example of the work done to prepare for writing such algorithms can be seen in Appendix C. This is a particularly interesting example as it broadly details the algorithm for dropping stitches. One challenge in writing these algorithms was to make sure that the integrity of the KnitGraph was maintained at all times no matter what algorithmic process was running at any given moment. The overall program was designed so that a user could easily switch between ”knitting” and ”unpicking” without disrupting the consistent state of the KnitGraph. In this respect, there was great care taken and continuous testing/debugging when writing the code (see Chapter 5).

4.4 The Sound Machine

Each stitch (node) in the graph represents a sound that is played as part of the program at various stages of it’s running. The SoundPlayer class was created to enable sounds to exist within the structure of the KnitGraph. For example, each time a new stitch is created, it is passed a new SoundPlayer object. This can then be played and updated depending on the requirements of the program. The SoundPlayer class was created using the Java Sound API sampled package. At it’s most basic, this allows you to play back and loop .wav files as Clips. It was decided to initially keep this part of the program simple since it was judged that more time/energy would be needed to implement the KnitGraph. The SoundPlayer loads a clip depending on a String command that is passed to it

via the constructor. Different sound files were pre-recorded and organised within the SoundPlayer into separate directories. The command passed to the SoundPlayer determines which directory a random clip is selected from.
The sounds recorded initially inspired by waulking songs of the Scottish Highlands [4]. Waulking songs were traditionally sang by women whilst they ”waulked” cloth as part of the weaving process. This involved beating the cloth to soften the wool. Singing these songs helped the women keep a steady rhythm and took the form of ”call and response”. Different elements of these songs were recorded as multiple individual tracks: ”Calling”, ”Response”, ”Clapping” and ”Whooping”. However, it was discovered that if the tracks were too complex e.g. contain a short melody, the resulting soundscape was overwhelming, unpleasant to listen to, and in no way evoked a sense of a waulking song. A slightly different approach was required. Shorter tracks were recorded using a range of delicate sounds such as those made when rubbing knitting needles together, or individual notes played on a kalimba (a type of thumb piano), and some categories were added such as ”Highlands”. These categories of sounds correspond with a specific directory in the SoundPlayer. Making new stitches (knitting), invokes a ”calling” or ”response” sound, where as removing stitches (unpicking), invokes a ”clapping” or ”whooping” sound. Sometimes there is overlap which gives an overall abstract impression of a waulking song. Once the KnitGraph is complete, it is then possible to ”play” it. ”Play” invokes all stitch-sounds to be played depending on the length of the edges. The length of the edges is determined by what command created the stitches it contains (some commands induce longer edges than others, as in real life). There is undoubtedly scope to be even more playful with this class and to more deeply explore the potential that the Java Sound API has to offer. More on this will be discussed in Chapter 7.

4.5 Future-Proofing & Limitations

An extra class (KnitFileSelector.java) was developed to allow for new KnitSpeak files to be added. A text file was created to input the new file-paths as Strings. This could then be read in and three random and unique paths added to an array which would then be passed to the GUI for further random selection. This also, allows the option to develop the GUI further and for the user to select from three randomly selected KnitSpeak files. ”Three” so that they can be organised in-line with the existing JButtons on the GUI. Complexity could be added by creating several text files containing file-paths to distinct types of KnitSpeak files e.g. lace-knit, cable-knit and so on. The user could then choose which type of knitted thing they want to ”play” before choosing from the randomly selected 3 options.
The program is a complete and working first version. It has been designed in such a way that improvements can be made and complexity added with relative ease. Current identified limitations to take into consideration are as follows:

The program can only handle knit commands that are present in the three selected KnitSpeak files. A whole remaining row must be unpicked before knitting can resume. Ideally, the program could unpick up to any stitch in the KnitGraph, and the knitting could pick up from that stitch once it resumes. This would be a more realistic representation of the knitting process. At the time of writing, a neat solution to this problem was yet to be found, but it was addressed in such a way that the program could continue development beyond this problem. The program was designed and built on a Mac OS, so it’s not guaranteed to work on other operating systems such as Linux and Windows.

Chapter 5

Testing

The approach to testing was varied and encapsulated both formal approaches such as the use of JUnit testing, and more ad-hoc approaches such as the inclusion of print statements in order to track a range of processes. Unit tests identified if there was a problem with the program as it progressed in the later stages of development, whereas print statements allowed the location and nature of the problem to be identified. The underpinning of both these approaches was the use of ad-hoc hand-drawn diagrams which helped to visually model the KnitGraph. This in turn helped to illustrate the state of the KnitGraph model at any given moment. By isolating parts of the overall diagram, the status of the model, as illustrated by printed data on the console, could be verified. It is important to note that this dynamic approach to testing was applied broadly across the whole program, and as new aspects of the program were developed, they were tested before moving on to the next stage of development, thus ensuring its robustness. More thorough testing was required of the KnitGraph.java class since it represents the main body of the program. For that reason, the focus here is on the KnitGraph itself, similar techniques were applied to other aspects of the program. By addressing the three main strands to this holistic approach to testing, further understanding can be gained.

5.1 Modelling the KnitGraph for Testing

An important part of the process of writing complex algorithms such as the ones that are intrinsic to the building and decomposing of the KnitGraph is visualising the problem. In order to do this, it has proved helpful to be able to draw out a visual representation of an expected KnitGraph. This typically was done on an ad-hoc basis, and was hand-drawn in the moment, as required. Figure 5.1 shows a more formal visualisation of a KnitGraph. In this example, the KnitSpeak file for the Small Heart pattern has been translated. Each stitch contains information about its index in the graph, and the command which created it. There is also information about how many stitches are created at the end of each row. All of these bits of data are essential to understanding the progress of building or decomposing a KnitGraph, and therefore also for understanding how to test and debug the program. Additional data can be added to the diagram as needed. For example, when testing the dropAStitch() method, it was important to track the turning stitches (stitch at the end of each row). Each stitch contains a boolean which states whether or not it is a turning stitch.

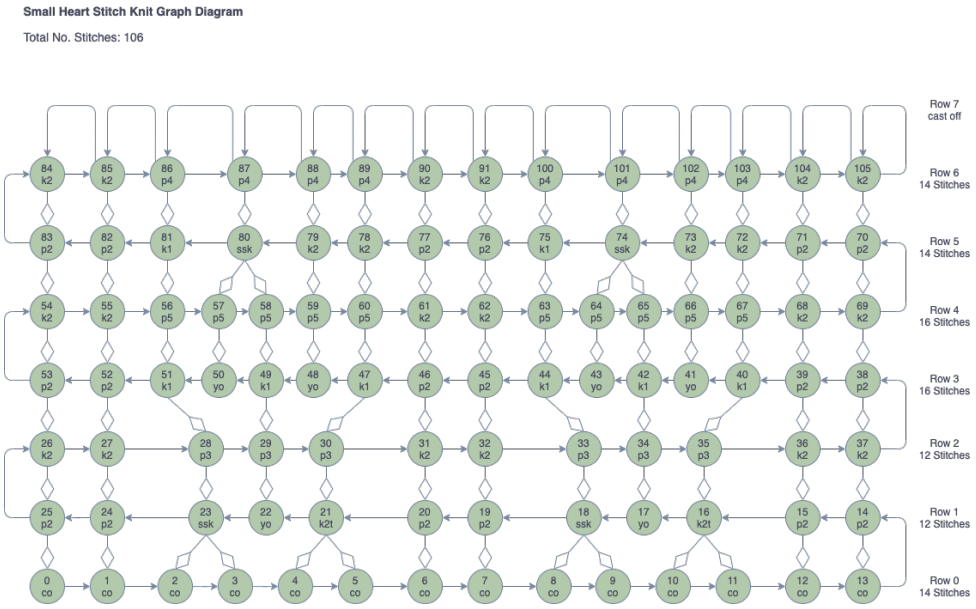


Figure 5.1: Small Heart Knit Graph Visualisation.

5.2 Informal Progressive Testing

In order to verify consistency in the status of the KnitGraph as it is being processed, print statements were coded to output relevant data to the console for comparing with data in the expected KnitGraph as illustrated in figure 5.1. Print statements were used at each stage of the build to verify the effectiveness of the work that had been done before moving on. This meant that any inconsistencies could be identified and located at the point of execution which improved the whole process of debugging. For an example see figure 5.2. Note how the statements track what stitches are being made and determines the consistent state of the KnitGraph as it switches from one process (knitting) to another (unpicking). It can be seen in this example that the knitting starts off where the unpicking was interrupted - stitch number 53.

5.3 Unit Testing using JUnit

Given the nature of the program, unit tests had their limitations which is why much of the testing was done in the way described above. However, for additional support and a cleaner way of reporting, some unit tests were written using JUnit. Again, due to its complexity, the focus was on testing the KnitGraph itself. The three main algorithms doKnitting, doUnpicking and dropAStitch were tested by checking against expected outcomes. For doKnitting, the method was run and the number of expected stitches compared. In order to test the other two methods, it was necessary to prepare a part-made graph. This was not possible using the doKnitting method as that was designed to knit the whole graph from start to finish unless interrupted. An additional method (knitPartOfGraph()) in the KnitGraph class had to be prepared which would create only part of the graph depending on an array of knit-commands and an integer that were passed to it. It was then possible to compare before and after figures for the doUnpicking method, and to directly compare which stitches had been dropped with those expected for the dropAStitch method.

5.4 Other Considerations

This is a multi-threaded program, there are threads in almost every aspect of it, swing-worker threads in the GUI, threads in the KnitGraph and threads (Clips) within the SoundPlayer class. Without careful attention, the number of threads could, and have, caused the heap to overload. This was a learning opportunity regarding how to execute threaded programs. The easiest way to test for these problems was to run the program. Issues were automatically reported by the IDE, and care was taken to close clips and join threads as necessary.



Figure 5.2: Debugging: Printing to the Console

Chapter 6

Evaluation

To understand how to evaluate this software, it is important to understand how to evaluate artworks in general. The question of how does one evaluate an artwork is age old [1], and really depends on your perspective of what *matters* and the context. In many ways, an artwork can be seen as part of the personal journey of the artist, something that the artist set out to do. In that sense, it can be evaluated based on how well that aim was achieved, but there is also scope for discussion and deeper understanding of the work that can help to evolve and progress it further. With this in mind and given the academic context of the work, it was decided to take the following two-stranded approach to the evaluation process: Feedback was sought from individuals and groups from two distinct backgrounds, those from an art background, and those from a software engineering background.



Figure 6.1: Women Waulking the Cloth. [4]

6.1 The Art Crit

A ”crit” is a formal way of receiving feedback on a given piece of work from peers, usually in an academic setting [23]. In a similar fashion to a focus group, the work is presented, and participants give feedback on how the work is perceived. A discussion then follows about how the work can be developed further. ”Crits” have a reputation for being often-times challenging for the artist since the feedback can be scathing [8]. This is deemed to be helpful in that the hope is that this approach inspires the artist to make improvements in response to extreme criticism. It was this approach that was used in evaluating this software-development project. The feedback given was qualitative in nature, and so in order to analyse it, notes were taken during and written reflections were prepared shortly after each ”crit”. The 6 participants that took part were either practising artists working in a range of fields such as sculpture, photography, textiles, all with a strong understanding of conceptual art, from an art-academic, or a curatorial background. None of the participants had a good understanding of computer programming. Due to current restrictions relating to the Covid global pandemic, all crits took place via Zoom, and the software was demonstrated through screen-sharing. Following is a reflection which consolidates all the feedback from the three separate sessions.

Every participant said that they found the project to be of interest. This was demonstrated directly, but also by the number of questions each person had about the software. There was a curiosity about something they did not understand (software development). Most participants (4/6) said that they would like to have a better understanding of the KnitGraph and suggested that this could perhaps be conveyed in a visual way either with supporting materials (diagrams for example), or as part of the GUI. One participant said that they could imagine the work as a ”soundtrack” to the experience of being in a gallery or museum and suggested that the GUI itself could have minimal aesthetics and provide function only to making the graph and its resulting sounds. Another participant said that the image on the GUI appeared to be ”cool” but that she felt a different image such as the one in Figure 6.1 would resonate with her more and give her a deeper sense of context regarding the work. This was qualified by saying that this image spoke more to her about the relationship between women, work, sound, textiles, history and tradition vs modernity, which are undoubtedly themes of the artwork. Half of the participants said that they would like to see a visualisation of the KnitGraph building itself on the screen. Two of the participants regularly work with adults with learning disabilities and described experiences where they could appreciate the complexities of simple tasks through the eyes of their clients. They were interested in the potential for code to address these complexities by breaking them down into smaller movements or actions. There was a discussion with the same two participants around the possibility to build a body of work, extra to the KnitGraph, that seeks to translate other domestic, traditionally female activities through code. One individual was interested in reversing the program to see if different sounds could be used to generate knitting patterns. Finally half of the participants expressed that they felt the software and concept would fit well in a museum setting as an interactive display, perhaps for children.

6.2 A Software Engineers Perspective

Feedback was sought out individually from a group of software engineers and developers. These were a mix of academics and those who had a number of years working in industry. The thing they had in common was that they had a higher level of experience than the developer of this piece of software. For that reason their feedback was deemed valuable in terms of how the design of the

software could be improved upon as well as to draw on their experience of working with users. Everyone in this group commented that they liked the GUI and it's background image. They could see clearly what was going on since it states that knitting is about to happen. Much of the discussion with this group was around presenting the work in a professional context. One suggestion on how to do this was to host the program online, and to include any supporting materials there too. In doing so, the hope would be that the programme could be easily understood by anyone accessing it without the need to clumsily share screens and give a verbal explanation each time. There was also some technical feedback, namely the suggestion to switch out .wav files for .mp3 as this would make the program smaller and more efficient to run. The Java Sound API only works for .wav files, so some further investigation would be required to adapt the program for .mp3.

6.3 Evaluation Summary

Having two different strands of enquiry proved to be useful. Interestingly, and counter to what was expected, the "artists" were really interested in seeing and learning about the code, whereas the "engineers" couldn't help but give feedback on the visual aspect of the work. Both groups of participants provided an insight into how they could appreciate the work on many levels. This points to a need from users to be able to engage with it in this way. That is to say, to see and understand the work as both a development project and a final piece of (art)work. The broad feedback suggested that the works value isn't limited to the user-experience with the GUI alone, but in also understanding the mechanisms which lie beneath. This is something to take into consideration when planning how the work could be presented. More specifically, the most popular suggestion on how to present the work was to be able to see a KnitGraph visualisation. However, some participants reported liking (or "loving" in one case) the GUI design and image, so this will require further investigation into how best to present the work. One option might be to present the KnitGraph visualisation separately from the original GUI rather than to have the visualisation as an integral part of it. In order to understand the best solution, it would be prudent to implement these changes and gain more feedback.
The two strands of enquiry also gave a rich and complimentary spectrum of feedback. The "artists" tended to seek *meaning* in the work and wanted to discuss if/how this was successfully portrayed, and if not, how it could be. They were also interested in how the work sits in the wider context of life/art. The "engineers" focused more on their own immediate response to the work, whether or not the GUI was intuitive to use, and how they felt the work would be perceived in a professional context e.g. how the work demonstrates the programmers skills in order to make them more employable. Overall the response to the project as a whole was that it was particularly interesting which gives reassurance that it has been a worthwhile endeavour and worthy of further development. Time permitting, it would have been beneficial to have made the most popular proposed changes and to get further feedback.

Chapter 7

Conclusion

It's not an easy task to ascertain the success of an artwork. It's appreciation is largely subjective. However, we have seen here that there is a solid context for including computer programming in the vernacular of acceptable modes for making art. There was a clear interest in the project from those working in the arts as well as people from a software engineering background. It could be said that this project bridges the gap between the two. There is at least *some* value in what has been achieved here. On the surface, what does this program actually do? You press a button and some sound comes out. It's fairly "useless" on the face of it, but it has prompted interesting discussions such as those around the complexity of simple human actions. And what more can you ask from an artwork?

The main thrust of this project was to develop a series of algorithms that could replicate the acts of knitting. Developing these algorithms was a challenge: throughout the whole process of building that specific part of the program, there was a constant need for debugging. The main difficulty was to be able to keep track of all the necessary variables and maintain a consistent state of the KnitGraph. This was overcome as described in Chapter 5. Other challenges included managing memory in a multi-threaded program and working with sound which was an unfamiliar medium.

Along with the feedback from participants of the evaluation process (Chapter 6), a sense of the potential of the project developed throughout its duration. Suggested future works can be divided into 3 categories, Presentation of the Product, Program Development and Project Development.

- **Presentation of the Product:** There are many options when it comes to presenting this work. Each option depends on a slightly different area of interest, and some options could work simultaneously. For example, the program could exist both in a physical space such as a gallery or museum, but also online. In both cases, it would be useful to provide supporting material, whether that is in the form of online or physical materials. This would be with the aim of making the program more accessible and for users to gain a deeper understanding of it. Materials might include for example, diagrams of the KnitGraph, information about the historical context, and teachings on how code can translate real-world activities. This would be particularly of interest if the destination for the program was a museum. Based on the feedback given as part of the evaluation, a future aim would be to create a visual element to the program that illustrates the KnitGraph as it grows/decomposes. This could be a responsive animation that appears on the screen in place of the current GUI image. The program was

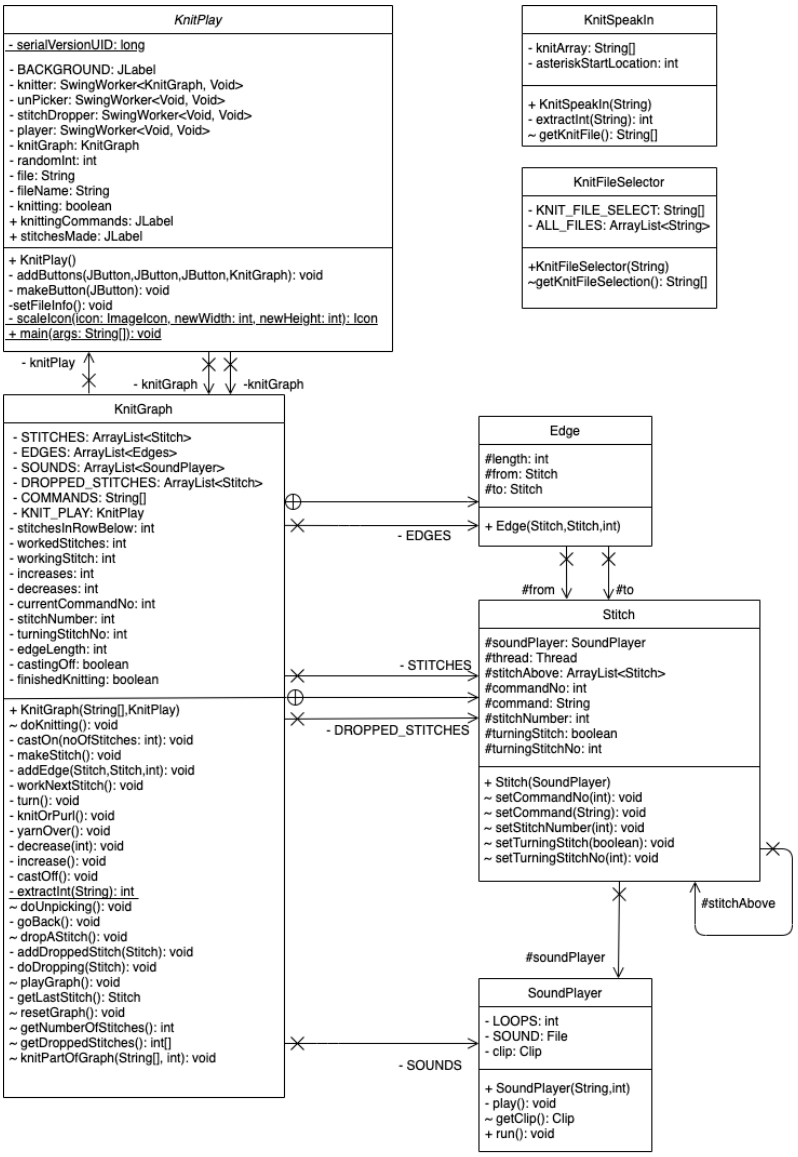
designed to allow for scalability. One way in which it achieved that was to make it easy to add more KnitSpeak files. In order to extend this idea, more options could be developed for the user. For example, the user could choose from a number of different categories such as ”cable knits”, ”lace knits” and so on. Within each category they could choose a specific pattern that they would like to ”knit”.

- **Program Development:** In order to further develop the program itself, the aim is to build up a larger repository of KnitSpeak files and to develop the code to allow for the inevitable increased complexity. This would likely involve adding cabled stitches which are stitches worked diagonally across a given number of stitches below. There is also a great deal of potential in developing the sound component of the program. Work could be done to create new and interesting soundscapes in collaboration with experienced individuals such as musicians and sound artists. A range of soundscapes could be developed that represent the different categories of KnitSpeak files. There is also potential with the Java Sound API to explore more deeply. For example, perhaps the KnitGraph could be translated into midi which could be fed into a synthesizer and sounds could be created dynamically that way.
- **Project Development** This program is seen as part of a wider project to explore the themes as discussed in Chapter 2. Two ways this project could be developed have been identified: One, develop a website as part of the broader project. This could act as a repository of information about the relationship between the arts, computers, electronic music, crafts and women. It could also include contemporary software art works that are relevant to this project and be a hub for people working in, or just interested in these fields of enquiry; Two, make a body of work that extends beyond this piece of software, that would explore other domestic/human actions that could be represented in code. Further research would be required for this, but areas of interest might be in gardening or playing a musical instrument for example. Or, sticking with fibre arts, it could extend to coding crochet, weaving and macramé.

Final Note: This project took place during the COVID global pandemic of 2020, so there were some limitations as to how things could proceed. The main ones being that no physical space could be accessed to display the work, no in-person evaluations could take place, and access was limited to a Mac computer only.

Appendix A

Class Diagram



Appendix B

KnitSpeak File Parsing

B.1 Example KnitSpeak files

Small Heart

Row 0: co 14 sts.
Row 1: *p2, k2tog, yo, ssk, repeat from *, p2.
Row 2: *k2, p3, repeat from *, k2.
Row 3: *p2, k1, yo, k1, yo, k1, repeat from *, p2.
Row 4: *k2, p5, repeat from *, k2.
Row 5: *p2, k2, ssk, k1, repeat from *, p2.
Row 6: *k2, p4, repeat from *, k2.
Row 7: cast off.

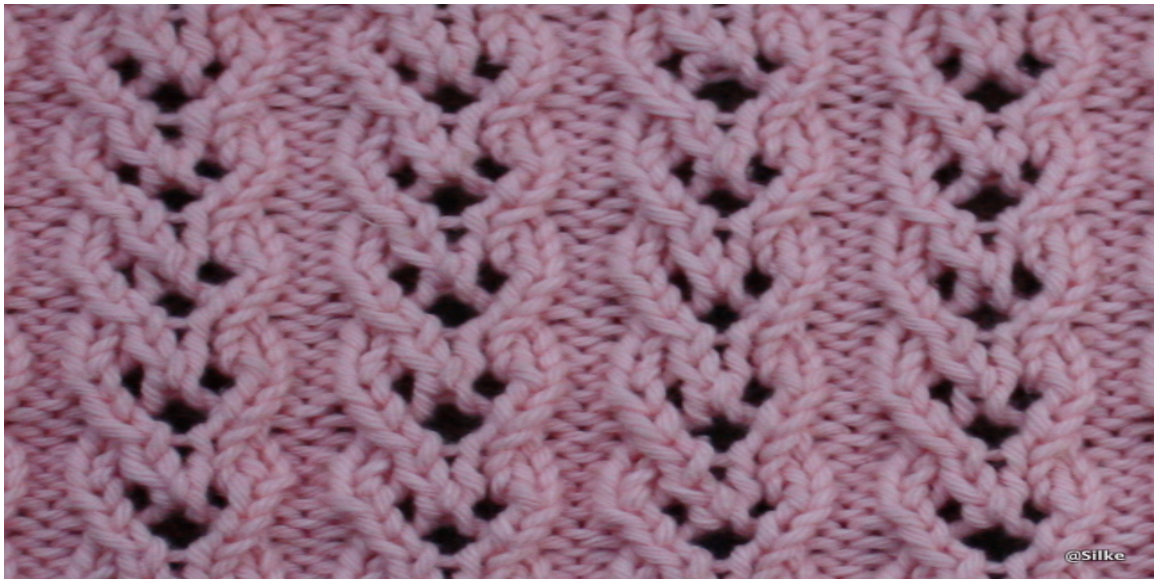


Figure B.1: Small Heart Sample [6]

Cherry Stitch

Row 0: co 25 sts.
Row 1: Purl.
Row 2: k3tog, *yo, kfb, k1, kfb, yo, k5tog, repeat from *, yo, kfb, k1, kfb, yo, sl1-k2tog-psso.
Row 3: Purl.
Row 4: k1 tbl, *yo, k2tog, k3, ssk, yo, k1 tbl, repeat from *, yo, k2tog, k3, ssk, yo, k1 tbl.
Row 5: Purl.
Row 6: k1 tbl, *yo, ssk, k3, k2tog, yo, k1 tbl, repeat from *, yo, ssk, k3, k2tog, yo, k1 tbl.
Row 7: Purl.
Row 8: k1 tbl, *yo, ssk, k3, k2tog, yo, k1 tbl, repeat from *, yo, k1 tbl, yo, k5tog, yo, k1 tbl, yo, k1 tbl.
Row 9: Purl.
Row 10: k1, *kfb, yo, k5tog, yo, kfb, k1, repeat from *, kfb, yo, k5tog, yo, kfb, k1.
Row 11: Purl.
Row 12: k1, *k1, ssk, yo, k1 tbl, yo, k2tog, k2, repeat from *, k1, ssk, yo, k1 tbl, yo, k2tog, k2.
Row 13: Purl.
Row 14: k1, *k1, k2tog, yo, k1 tbl, yo, ssk, k2, repeat from *, k1, k2tog, yo, k1 tbl, yo, ssk, k2.
Row 15: Purl.
Row 16: k3tog, *yo, k1 tbl, yo, k1 tbl, yo, k1 tbl, yo, k5tog, repeat from *, yo, k1 tbl, yo, k1 tbl, yo, k1 tbl, yo, sl1-k2tog-psso.
Row 17: Purl.
Row 18: cast off.

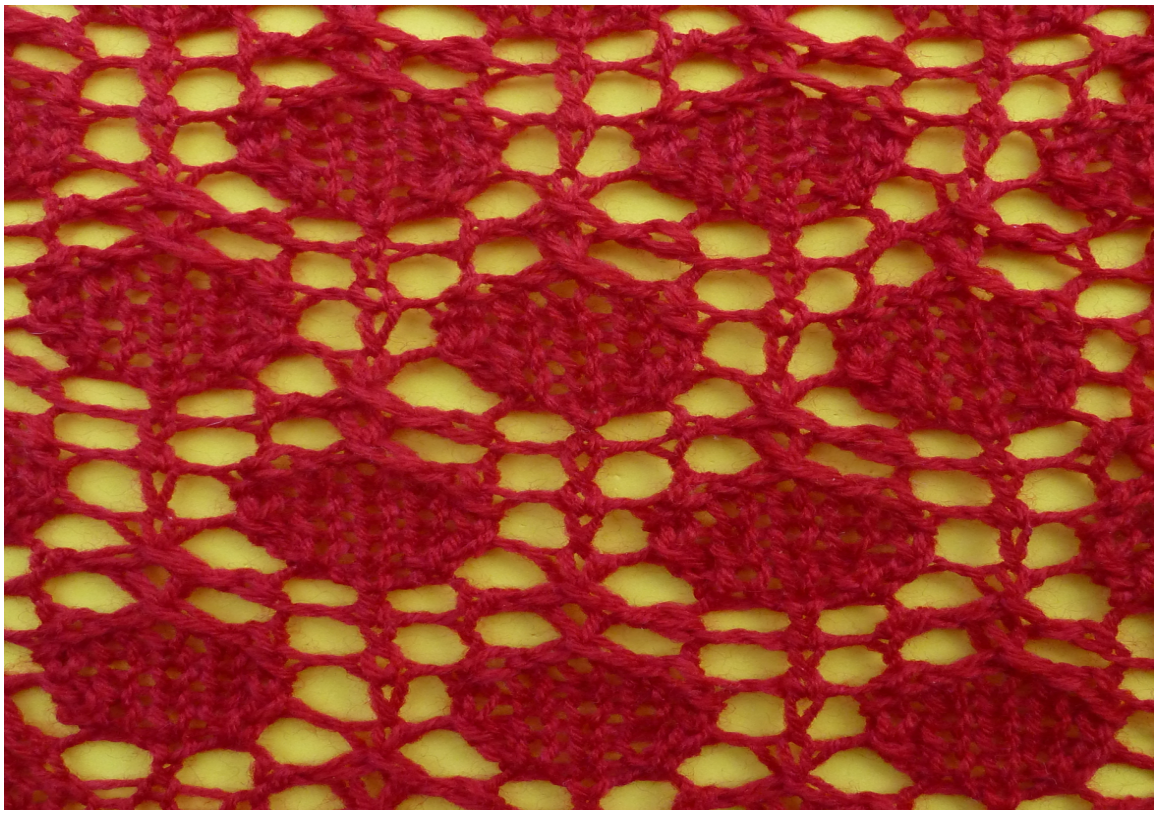


Figure B.2: Cherry Stitch Sample [6]

B.2 Pseudocode and Workings for KnitSpeakIn

Algorithm 1 Algorithm for Repeated Knit-Commands

Given an ArrayList "commands"
$asteriskFound = false, reps = 0, asteriskStartLocation = null, count = 0.$
for all i such that $0 \leq i \leq commands.size$ **do**
  if commands at i contains '*' **then**
    if $asteriskFound = false$ **then**
      $asteriskFound = true, asteriskStartLocation = i$
      remove '*' from commands at i
    else if commands at i contains an integer n **then**
      $reps = n$
      while $count < reps$ **do**
        $sub = i - 1$
        add commands at sub to commands at i
        $sub--, count++$

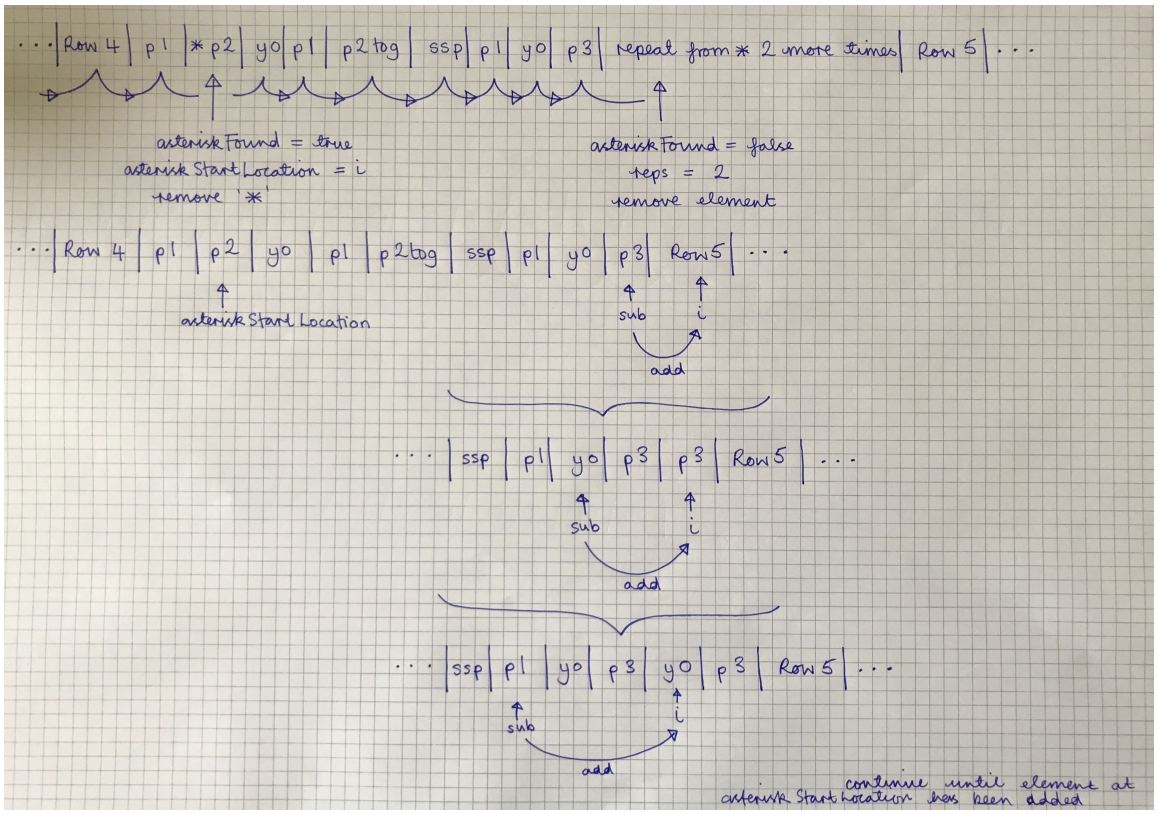


Figure B.3: Algorithm Workings

Appendix C

Sample Workings for KnitGraph

C.1 Creating the dropAStitch() Algorithm

Here is a broad overview of this particular algorithm. A more detailed/nuanced version can be viewed in the code itself. This algorithm takes place in the following stages: Locate the stitches that will be dropped; Replace all relevant edges with a single (longer) edge to fill the space where the stitch will be removed; Remove all dropped stitches from the KnitGraph.

Locate Stitches to be Dropped (See example in Figure C.1)
Note: Each ”worked” stitch contains a reference/s to the stitch/es above it.

First, find the stitch that would be dropped at the point of a call of dropAStitch(), a call made between the start and end of ”knitting” the KnitGraph. This stitch would be the stitch before the last worked stitch in the row below the current row. Add a reference to that stitch to a list of DROPPED_STITCHES.
Second, find the stitch/stitches that contain a reference to that stitch in their stitchAbove list, add a reference to it/them to the DROPPED_STITCHES list.
Repeat these two stages, until no references to previously dropped stitches can be found in any stitches stitchAbove list. Return a list of stitches to be dropped: DROPPED_STITCHES.

Replace Edges and Remove Stitches (See example in Figure C.2)
Iterate over the list of DROPPED_SITCHES and replace each stitch’s two edges with a single longer edge. The length of the new edge is determined by the number of horizontally adjacent stitches to be removed. We can then iterate over the list of DROPPED_STITCHES and remove those stitches referenced there from the KnitGraph.

Some Exceptions and Things to Note
If a dropped stitch is a ”yo” stitch, there are no stitches below it to drop. If the KnitGraph is in the process of casting-off, then no stitches can be dropped (this was a design decision. In real life you could feasibly drop stitches whilst casting-off). Finally, if a call of dropAstitch() is made once a row has been completed, or during the cast-on row, no stitches will be dropped.

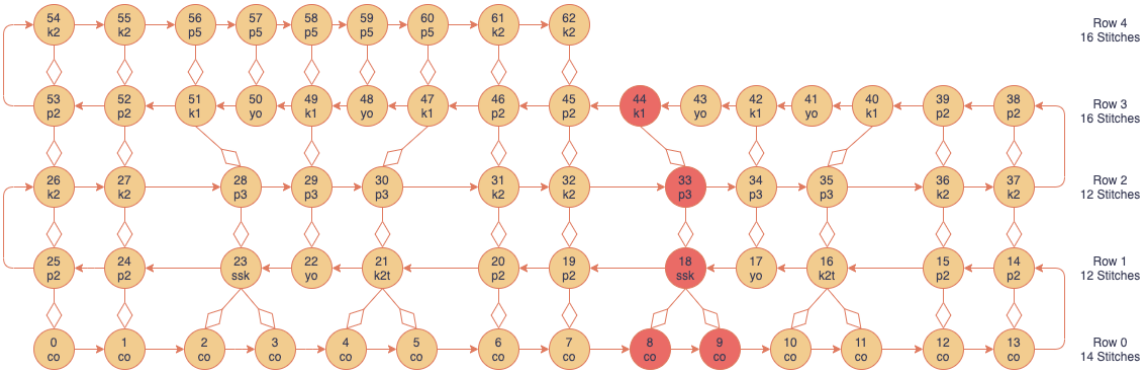


Figure C.1: Finding which stitches will be dropped (best viewed in colour).

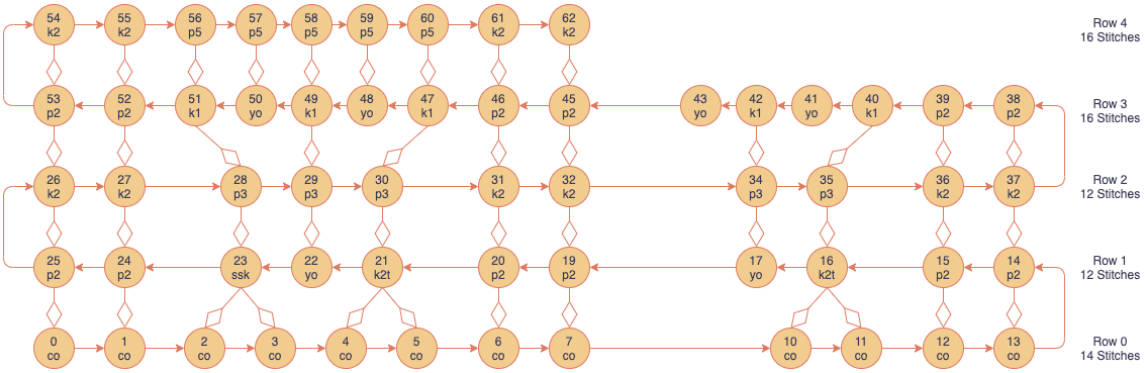


Figure C.2: Visualisation of KnitGraph once edges have been replaced and dropped stitches have been removed (best viewed in colour).

Appendix D

Ethics Checklist

School of Computing Science
University of Glasgow

Ethics checklist form for assessed exercises (at all levels)

This form is only applicable for assessed exercises that use other people ('participants') for the collection of information, typically in getting comments about a system or a system design, or getting information about how a system could be used, or evaluating a working system.

If no other people have been involved in the collection of information, then you do not need to complete this form.

If your evaluation does not comply with any one or more of the points below, please contact the Department Ethics Committee for advice.

If your evaluation does comply with all the points below, please sign this form and submit it with your assessed work.

---

1. Participants were not exposed to any risks greater than those encountered in their normal working life.
   Investigators have a responsibility to protect participants from physical and mental harm during the investigation. The risk of harm must be no greater than in ordinary life. Areas of potential risk that require ethical approval include, but are not limited to, investigations that occur outside usual laboratory areas, or that require participant mobility (e.g. walking, running, use of public transport), unusual or repetitive activity or movement, that use sensory deprivation (e.g. ear plugs or blindfolds), bright or flashing lights, loud or disorienting noises, smell, taste, vibration, or force feedback

2. The experimental materials were paper-based, or comprised software running on standard hardware.
   Participants should not be exposed to any risks associated with the use of non-standard equipment: anything other than pen-and-paper, standard PCs, mobile phones, and PDAs is considered non-standard.

3. All participants explicitly stated that they agreed to take part, and that their data could be used in the project.
   If the results of the evaluation are likely to be used beyond the term of the project (for example, the software is to be deployed, or the data is to be published), then signed consent is necessary. A separate consent form should be signed by each participant.

   Otherwise, verbal consent is sufficient, and should be explicitly requested in the introductory script.

4. No incentives were offered to the participants.
   The payment of participants must not be used to induce them to risk harm beyond that which they risk without payment in their normal lifestyle.

5. No information about the evaluation or materials was intentionally withheld from the participants.
   Withholding information or misleading participants is unacceptable if participants are likely to object or show unease when debriefed.

6. No participant was under the age of 16.
   Parental consent is required for participants under the age of 16.

7. No participant has an impairment that may limit their understanding or communication.
   Additional consent is required for participants with impairments.

8. Neither I nor my supervisor is in a position of authority or influence over any of the participants.
   A position of authority or influence over any participant must not be allowed to pressurise participants to take part in, or remain in, any experiment.

9. All participants were informed that they could withdraw at any time.
   All participants have the right to withdraw at any time during the investigation. They should be told this in the introductory script.

10. All participants have been informed of my contact details.
    All participants must be able to contact the investigator after the investigation. They should be given the details of both student and module co-ordinator or supervisor as part of the debriefing.

11. The evaluation was discussed with all the participants at the end of the session, and all participants had the opportunity to ask questions.
    The student must provide the participants with sufficient information in the debriefing to enable them to understand the nature of the investigation.

12. *All the data collected from the participants is stored in an anonymous form.*
    All participant data (hard-copy and soft-copy) should be stored securely, and in anonymous form.

Module and Assessment Name MSc Development Project for IT+

Student's Name Emily Quimby

Student's Registration Number 9906376O

Student's Signature

Date 14/09/20

Bibliography

[1] ART; Is It Art? Is It Good? And Who Says So? https://www.nytimes.com/1997/10/12/arts/art-is-it-art-is-it-good-and-who-says-so.html. (Accessed: 27-08-20).

[2] Olia Lialina's Most Famous Net Art Piece Turns 15: My Boyfriend Comes Back From The War, the death of the frameset and other Web 1.0 magic. *Vice*, August 2011.

[3] Chelsea Battell. Domain specific language for modular knitting pattern definitions: Purl. *ArXiv*, abs/1606.08708, 2016.

[4] BBC. BBC Bitesize. https://www.bbc.co.uk/bitesize/clips/z3np6fr. (Accessed: 13-08-20).

[5] Miguel Benavides. Cybernetic Serendipity 50th Anniversary. January 2018. Available online at https://www.studiointernational.com/index.php/cybernetic-serendipity-50th-anniversary (Accessed 24-06-2020).

[6] JC Briar. Stitch-Maps. http://stitch-maps.com. (Accessed: 24-06-20).

[7] James Bucanek. *Learn Objective-C for Java Developers*, pages 353–402. Apress, New York, 2009.

[8] Dr Peter Day. The Art Group Crit. How do you make a Firing Squad Less Scary? *brightONLINE student Literary Journal*, 18, March 2012.

[9] Alison Drain. Ben Laposky: Visual Music. *Symmetry Magazine*, January 2007.

[10] Francesca Elliott. WEAVING NUMBERS: THE JACQUARD LOOM AND EARLY COMPUTING. https://blog.scienceandindustrymuseum.org.uk/weaving-numbers-jacquard-loom-early-computing/. (Accessed 24-06-2020).

[11] W. B. Fritz. The women of eniac. *IEEE Annals of the History of Computing*, 18(3):13–28, 1996.

[12] Cassidy George. How Knitting Became Cool. https://www.bbc.com/culture/article/20200630-how-knitting-became-cool. (Accessed 01-08-2020).

[13] Sarah Gottesman. 10 Pioneering Textile Artists, from Sheila Hicks to Nick Cave. https://www.artsy.net/article/artsy-editorial-10-textile-artists-who-are-pushing-the-medium-forward, October 2016. (Accessed 24-06-2020).

[14] Marie Hicks. *PROGRAMMED INEQUALITY: How Britain Discarded Women Technologists and Lost its Edge in Computing*. The MIT Press, Cambridge, Massachusettes, USA, 2017.

[15] Megan Hofmann, Lea Albaugh, Ticha Sethapakadi, Jessica Hodgins, Scott E. Hudson, James McCann, and Jennifer Mankoff. Knitpicking textures: Programming and modifying complex knitted textures for machine and hand knitting. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*, UIST '19, page 5–16, New York, NY, USA, 2019. Association for Computing Machinery.

[16] Sara Landeau. 4 Women Who Pioneered Electronic Music. https://www.adorama.com/alc/women-electronic-music-producers-you-should-know. (Accessed 24-06-2020).

[17] Lindive Matlali. Knitting is Coding. https://tedxbeaconstreet.com/videos/knitting-is-coding/, 2017. TedX video (Accessed 24-06-2020).

[18] Ted Mills. Hear the First Musical Composition Created by a Computer: The Illiac Suite (1956). `http://www.openculture.com/2019/02/hear-the-first-composition-created-by-a-computer.html`. (Accessed: 24-06-2020).

[19] F. Richard Moore. Dreams of Computer Music: Then and Now. *Computer Music Journal*, 20(1):25–41, 1996.

[20] Markku Reunanen. Times of Change in the Demoscene: A Creative Community and Its Relationship with Technology. 2017.

[21] Siobhan Roberts. Christopher Strachey's Nineteen-Fifties Love Machine. *The New York Times*, 2017. Available online at `https://www.newyorker.com/tech/annals-of-technology/christopher-stracheys-nineteen-fifties-love-machine` (Accessed 24-06-2020).

[22] Siobhan Roberts. 'Knitting Is Coding' and Yarn Is Programmable in This Physics Lab: For Elisabetta Matsumoto, knot theory is knit theory. *The New York Times*, 2019. Available online at `https://www.nytimes.com/2019/05/17/science/math-physics-knitting-matsumoto.html` (Accessed 24-06-2020).

[23] Sarah Rowles. *Art Crits 20 Questions A Pocket Guide*. Q-Art London, London, UK, 2014.

[24] Richard Rutt. *A History of Hand Knitting*. Batsford Ltd, London, UK, 1987.

[25] Andrea K. Scott. Futurism: Cory Arcangel plays around with technology. *The New York Times*, May 2011.

[26] Margot Lee Shetterly. *Hidden Figures: The Untold Story of the African American Women Who Helped Win the Space Race*. Wiliam Collins, London, UK, 2017.

[27] Emily Temple. 40 New Feminist Classics You Should Read. https://lithub.com/40-new-feminist-classics-you-should-read/. (Accessed 24-06-2020).

[28] V&A. A History of Computer Art. 2016. Available online at `http://www.vam.ac.uk/content/articles/a/computer-art-history/` (Accessed: 24-06-20).

[29] Joanna Walewska. The Art of Computing as Frieder Nake's Response to the Problem of "Mechanized Mental Labor": The Early Debates Revisited. *Leonardo*, 52:54–59, February 2019.